Obtain an X.509 Certificate

- Requires both a Public (.crt ,.cer) and Private Key (.p12,.pfx).
- See script below for certificate requirements.
- Edit the DNSName and Subject
- Edit the NotBefore and NotAfter values
- The .CER and .PFX file will be in your Documents folder.
- We create the certificate in the CurrentUser store due to permissions issues. It gets exported and then imported to the LocalMachine store. If we are testing this script and not running it from Intune (in the SYSTEM context), we need to grant ourselves access to it.

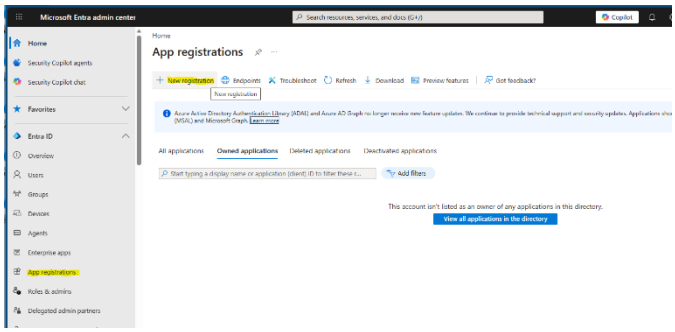
```
$params = @{
    Type = 'Custom'
    DnsName = 'ianbaxter.onmicrosoft.com'
    CertStoreLocation = 'Cert:\CurrentUser\My'
    KeyExportPolicy = 'Exportable'
    KeySpec = 'Signature'
    Subject = 'CN=Data Collect'
    KeyUsage = "DigitalSignature"
    KeyAlgorithm = 'RSA'
    KeyLength = 2048
    HashAlgorithm = 'SHA256'
    NotBefore = (Get-Date)
    NotAfter = (Get-Date).AddMonths(24)
    }
#
# Create the Self-Signed Certificate
New-SelfSignedCertificate @params
# Locate and load the certificate we just created
$cert = Get-ChildItem -Path "Cert:\CurrentUser\My" | Where-Object {$_.Subject -Match
"Data Collect"}
# Get thumbprint
$thumbprint = ($cert | Select-Object Thumbprint, FriendlyName).Thumbprint
#
# Export the Public Key (.CER)
Export-Certificate -Cert $cert -FilePath "$env:USERPROFILE\Documents\Data_Collect.cer"
#
# Create the password we need for the Private Key
# Enter password here
$MyPassword = "SomePassword"
$mypwd = ConvertTo-SecureString -String $MyPassword -Force -AsPlainText
#
# Export the Private Key (.PFX)
Export-PfxCertificate -Cert $cert -FilePath
"$env:USERPROFILE\Documents\Data_Collect.pfx" -Password $mypwd
#
# Delete by ThumbPrint
Remove-Item -Path "Cert:\CurrentUser\My\$thumbprint" -DeleteKey
```

Create and secure an App Registration

Launch the Entra ID portal

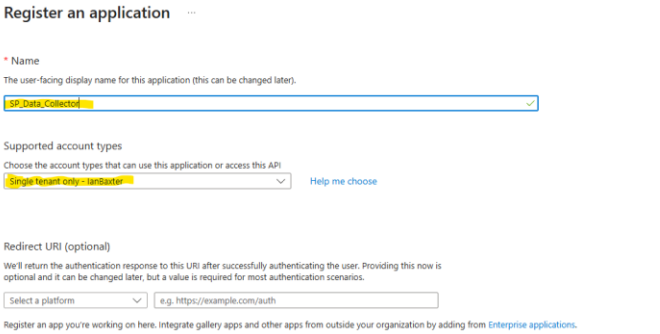
Choose “App Registrations” from the left menu bar.

Click “New Registration” from the top menu bar.



Give your Service Principal a name.

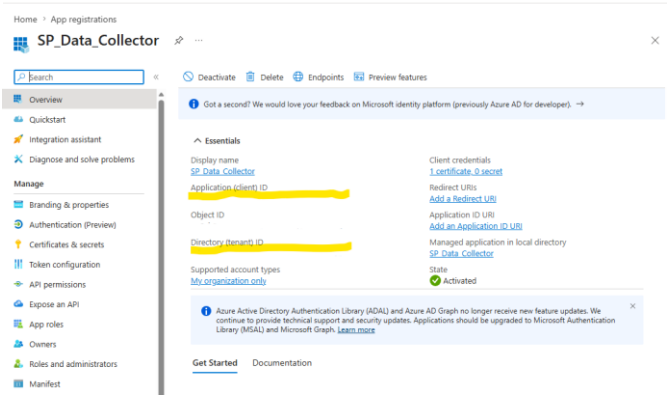
Unless you know you need it, you should probably just allow “Single Tenant Only” for the Account Types.



You should make a note of the following properties:

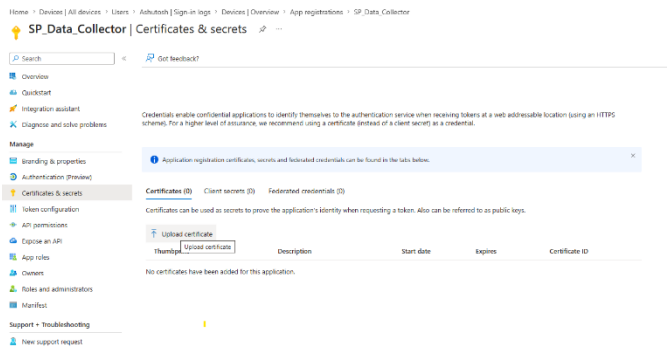
- Application (client) ID
- Directory (tenant) ID

These will be needed to test the Custom Compliance script.



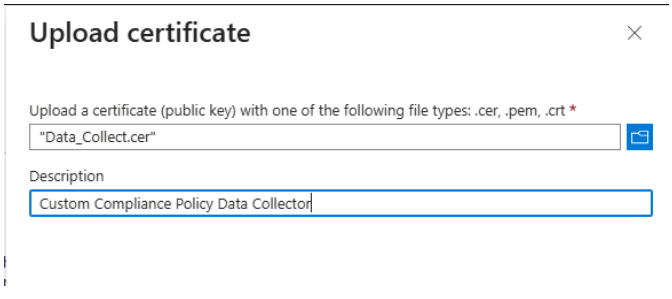
Select “Certificates and Secrets” from the left menu bar.

Select the “Certificates” tab and click the “Upload Certificate” button.



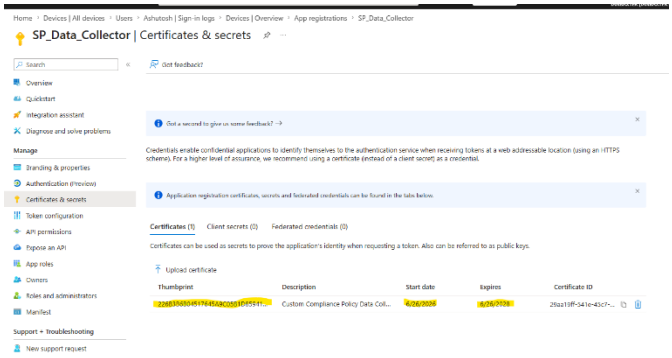
Select the file containing the public key (.crt or .cer).

Give the Descriptor some data to properly identify the certificate.



Make a note of the thumbprint so you can verify the private key when you install it on the test device.

Verify that you have the right dates on the certificate. (Note: You will see an icon to the right of the certificate if it has expired.)

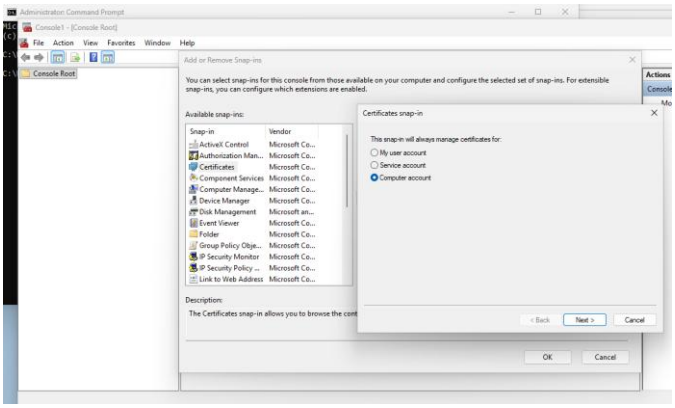


Install the Private Key on the test machine

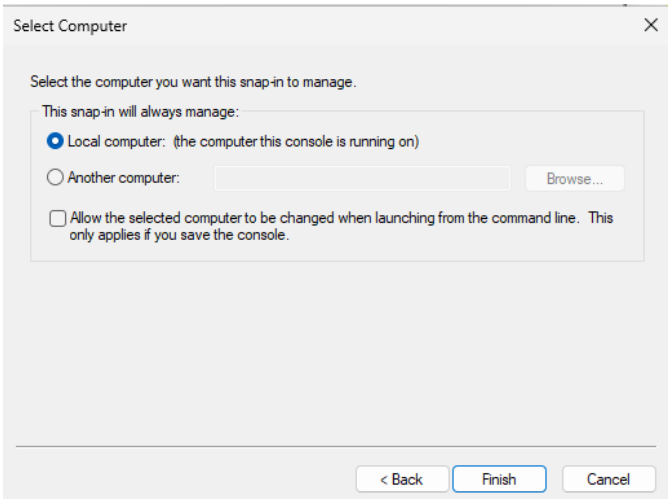
Since this is a test machine, you will be running the script as a user. The LocalMachine certificate store restricts users from reading the private key information from the certificate.

You need to have administrative rights to perform this step. Use the Windows search to find the icon for “CMD”. Right-click on the icon and choose “Run as Administrator”. Using the command prompt, run “MMC.EXE”, Select “File”, “Add/Remove Snapin…” and add the “Certificates” snap in.

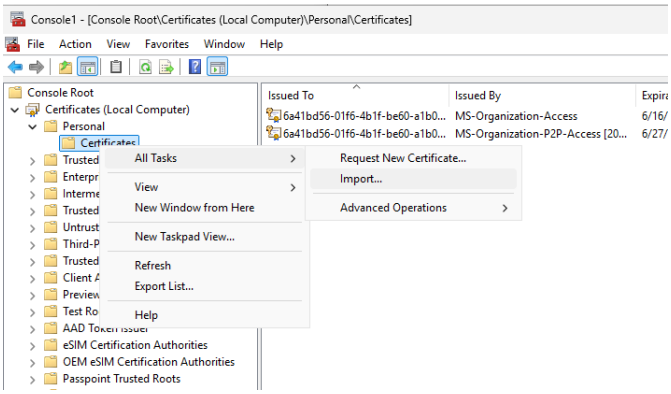
Select “Computer Account” and click “Next.’



Select “Local Computer” and click “Finish”.



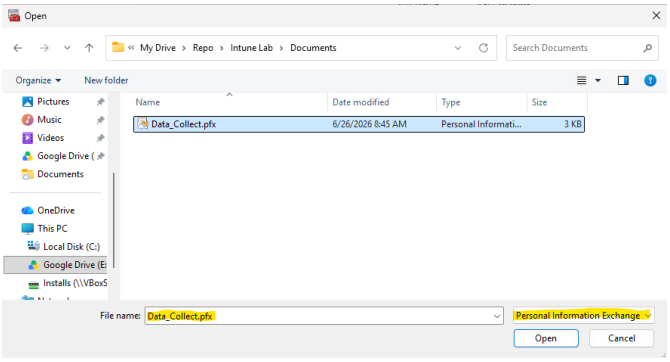
Locate the \Personal\Certificates folder in the left pane. Right-click, chose “All Tasks” and “Import…”



Set the file type dropdown to “Personal Information Exchange”.

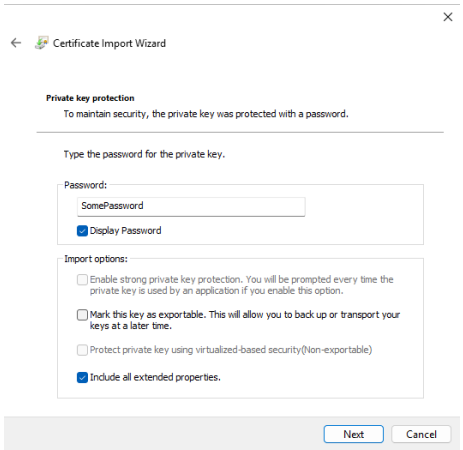
Browse to your private key file and select it.

Click “Open”.

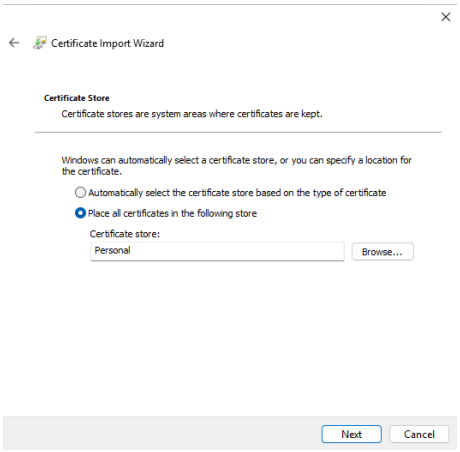


Remember the password you used to create the private key file? Enter it in the password text box.
If you don’t have the password, you need to get a new certificate and start over.

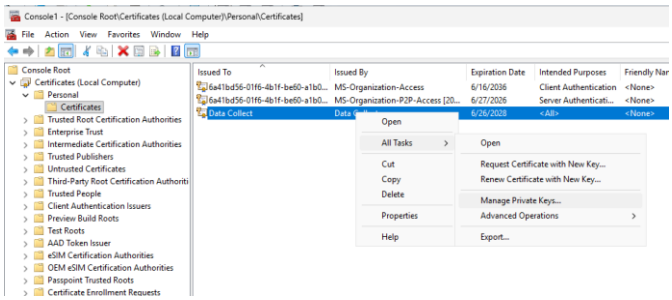
Do not select the option to make the certificate exportable.



Verify the certificate store is “Personal” and click “Next”.



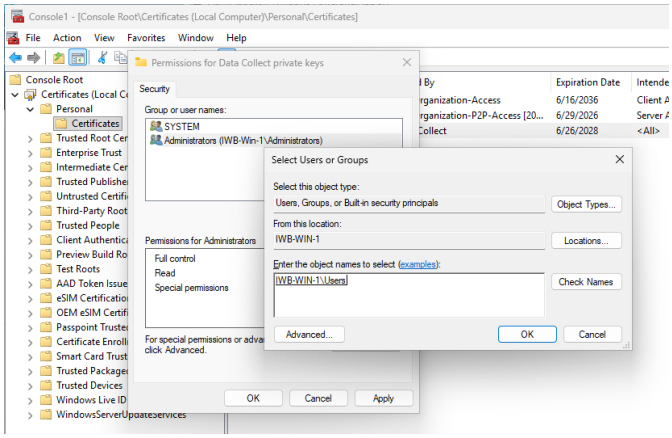
Right-click on the certificate, choose “All Tasks” and “Manage Private Keys…”.



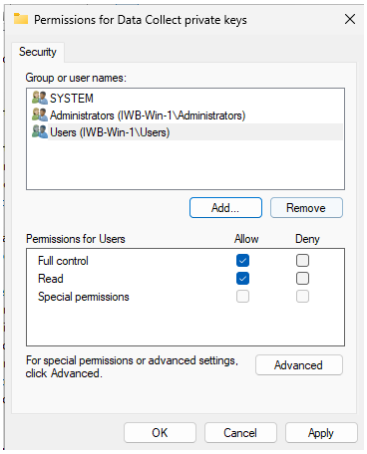
A permissions dialog will pop up. Click “Add”.

Enter “Users” in the bottom box, click “Check Names” to ensure the group or account can be resolved.

Click “OK”.



Verify the group or account has at least “Read” and “Full Control” access. You need at least “Read”, but “Full Control” will be needed to delete the certificate later.



Alter the Inventory Script and test (save JSON)

We are going to use a Windows PowerShell script for testing (Custom_Inventory.ps1). You will need to run the script using administrative access.

Enter “Powershell_ise” in the Windows Search box. Locate the icon for “Windows PowerShell ISE 5.1”, right-click on it and choose “Run as Administrator”.

You will need to run “Set-ExecutionPolicy -Scope LocalMachine -ExecutionPolicy Unrestricted ” in the console.

The first few lines of the script are configuration variables.

Populate:

- $ApplicationID
- $TenantID
- $CertName
- $LogFile

```
# Service Principal Data (Required for Token)
$ApplicationID = "<Application/Client ID>"
$TenantID = "<Tenant ID>"
$CertName = "Data Collect"
#
# Log Analytics Data (for JSON submission)
$DcrImmutableId = "<DCR Immutable ID>"
$DceURI = "<DCE Data Ingestion URL>"
$streamName = "<Stream Name from DCR JSON>"
#
# Script logging
$LogFile = "<Log File Path>"
#
```

The next block of code are the functions that provide functionality to find the correct certificate and then use it to obtain an access token via OAuth2 after authenticating as the Service Principal.

At the beginning of the script, we use those functions. The script will not run unless a valid certificate is found.

```
#
# BEGIN SCRIPT
#
#
# Get the first valid certificate that matches $CertName
$cert = Get-ValidCertificate -CertName $CertName
# If one was found
if ($cert) {
    # Get the thumbprint
    $thumbprint = (Get-ChildItem -Path "Cert:\LocalMachine\My" | Where-Object {$_.Subject -Match $CertName}).Thumbprint
    #
    # BEGIN INVENTORY
```

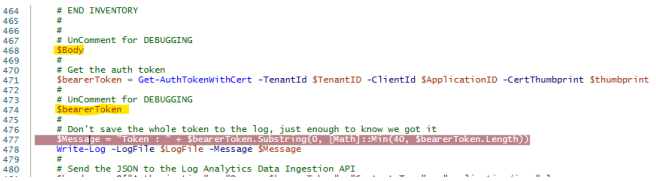
Customize the Inventory block to collect the data you want to report via JSON upload. If you are reporting for both Windows and MacIntosh, remember that your JSONs must have identical structures and plan accordingly.

```
    #
    # BEGIN INVENTORY
    #
    #
    #
    # Get the ManagedDeviceID and Name from Intune enrollment
    try {
        # If the device is enrolled
        if (@(Get-ChildItem HKLM:SOFTWARE\Microsoft\Enrollments\ -Recurse |
            # Get the source for the managed device information
```

Use the code example to create the structure we are going to export to JSON.

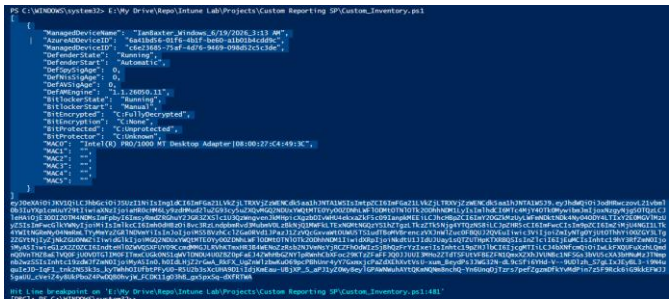
```
    #
    # Compile all the settings in a form we can easily convert to JSON
    $Inventory = New-Object System.Object
    $Inventory | Add-Member -MemberType NoteProperty -Name "ManagedDeviceName" -Value $ManagedDeviceName -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "AzureADDeviceID" -Value $AzureADDeviceID -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "ManagedDeviceID" -Value $ManagedDeviceID -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "DefenderState" -Value $Defender_State -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "DefenderStart" -Value $Defender_Start -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "DefSpySigAge" -Value $Defender_SpySigAge -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "DefNisSigAge" -Value $Defender_NisSigAge -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "DefAVSigAge" -Value $Defender_AVSigAge -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "DefAMEngine" -Value $Defender_AMEgine -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "BitlockerState" -Value $Bitlocker_State -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "BitlockerStart" -Value $Bitlocker_Start -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "BitEncrypted" -Value $BitEncrypted -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "BitEncryption" -Value $BitEncryption -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "BitProtected" -Value $BitProtected -Force
    $Inventory | Add-Member -MemberType NoteProperty -Name "BitProtector" -Value $BitProtector -Force
    foreach ($thiskey in ($NetHash.Keys | Sort-Object)) {
        $Name = "MAC" + $thiskey.ToString()
        $Value = $NetHash[$thiskey]
        $Inventory | Add-Member -MemberType NoteProperty -Name $Name -Value $Value -Force
    }
    #
    # Convert to JSON for sending the data to Log Analytics Workspace
    $body = $Inventory | ConvertTo-Json
    #
    # END INVENTORY
```

After the inventory data has been converted to JSON, we have a block where breakpoints can be set to allow debugging.



Set a breakpoint on this line of code.

If you did everything correctly, you would see the JSON contents followed by the access token you received.



If not, you need to fix something either with your certificate or with your inventory code.

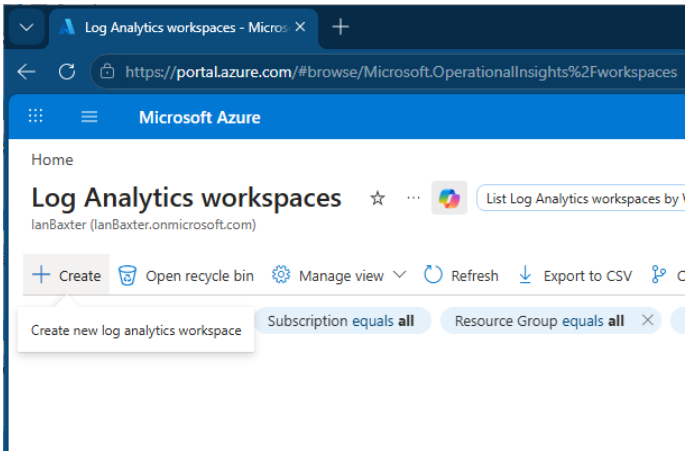
Copy the JSON (only the parts between “{“and “}”, and with those characters and save it as a text file. The “[“and “]” bracket make it a JSON array, so do NOT include those. Save this data to a text file with the “.json” extension. You will need it later.

Comment out the highlighted lines of code and remove the breakpoint.

Create the Log Analytics Workspace

Launch the Azure Portal, search for Log Analytics workspaces.

Click on “Create”.

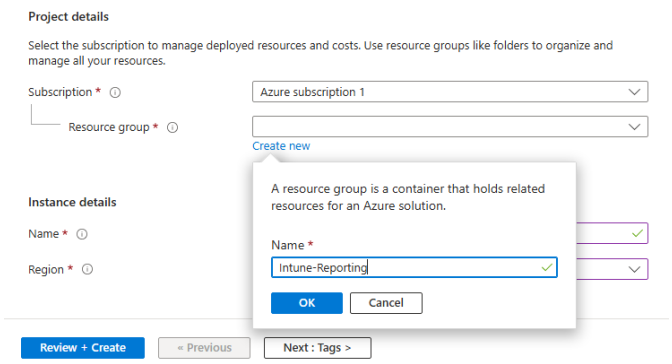


You may need to create a Resource Group. If so, click the “Create New” option.

Populate the Name and Region, click “Review + Create”.

When the validation passes, click “Create”.

Click the “Go to Resource” option.



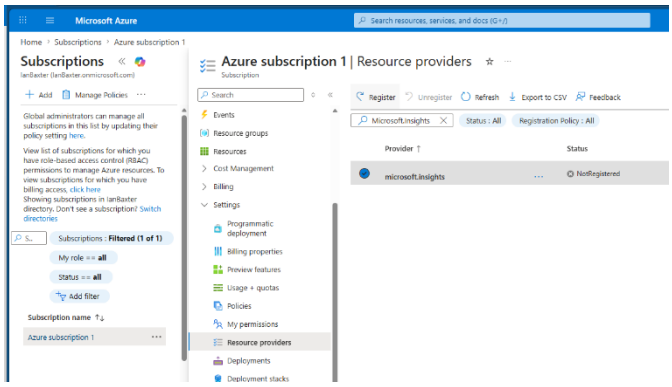
Register Microsoft Insights resource provider

In the Azure Portal, search for “Subscription” and load the applet.

Select the correct subscription. Click on “Resource Providers”, search for “Microsoft.Insights”.

Select it and click the “Register” option.

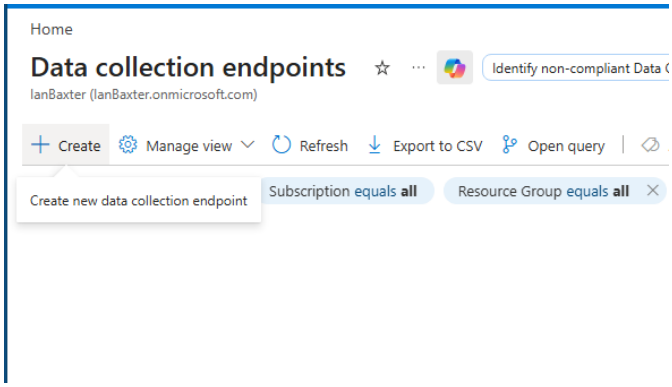
Wait for the notification that the registration has completed.



Create a DCE (Data Collection Endpoint)

Open the Azure Portal, search for “Data Collection Endpoints” and launch the applet.

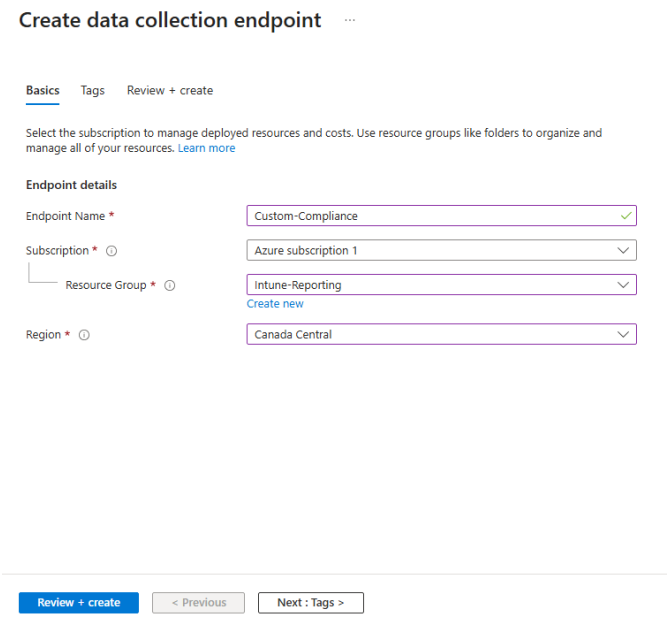
Click the “Create” option.



Populate the “Endpoint Name”, select the appropriate Subscription, Resource Group and Region.

The Region should be the same as your Tenant and must match the values for the DCR and Log Analytics workspace.

Click “Review and Create.



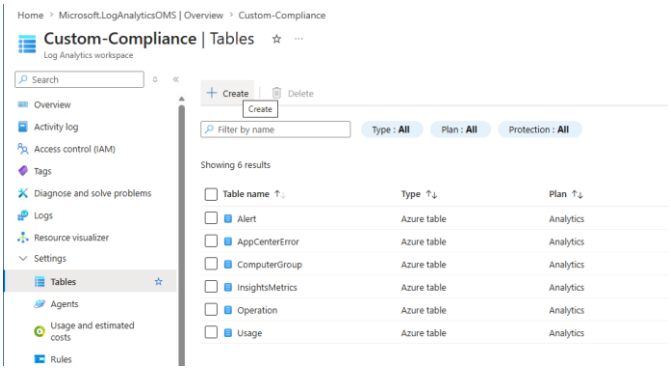
Create the Log Analytics Table and DCR from JSON

Let me caution you here. You need to do this once; do it right and fully understand what you are doing at this step. It seems simple from all the Internet documentation, but it really is not and is poorly documented.

If you make a mistake, you cannot delete the old table and DCR. The data that underlies them can persist for up to 90 days. The data for the table or DCR does not get deleted and will re-populate in any new one you recreate. The only other solution is to create one with a different name.

With the log analytics workspace open, expand the “Settings” and select “Tables”.

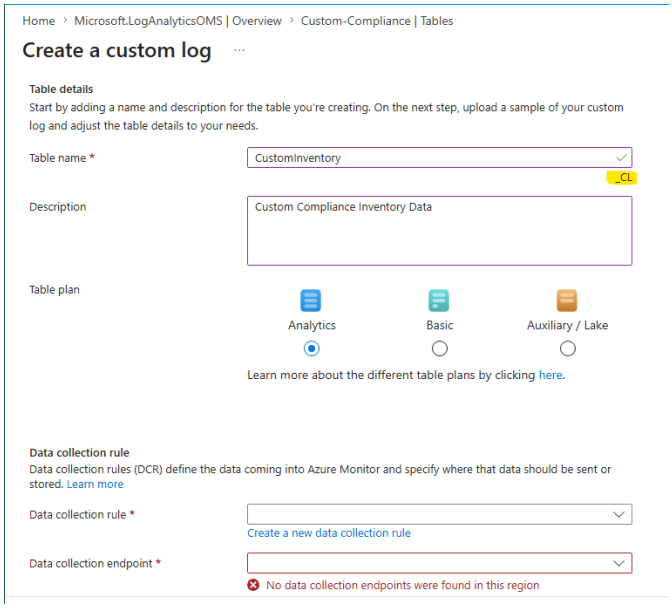
Click the “Create” option.



The table name must be the same as the table you will configure in the script with a few differences. The script uses a stream name which is “Custom-<TableName>_CL”. The “_CL” gets appended to the table name for “Custom Log” in Log Analytics, and the “Custom-“ gets added by the DCR to form the data stream name.

Enter a description.

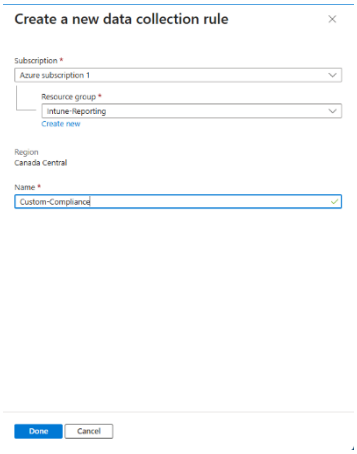
You should not have a DCR yet, so click the “Create a new data collection rule” option.



Choose your subscription and resource group, give the DCR a name.

Click “Done”.

Select the Data Collection Endpoint you created earlier, click “Next”.

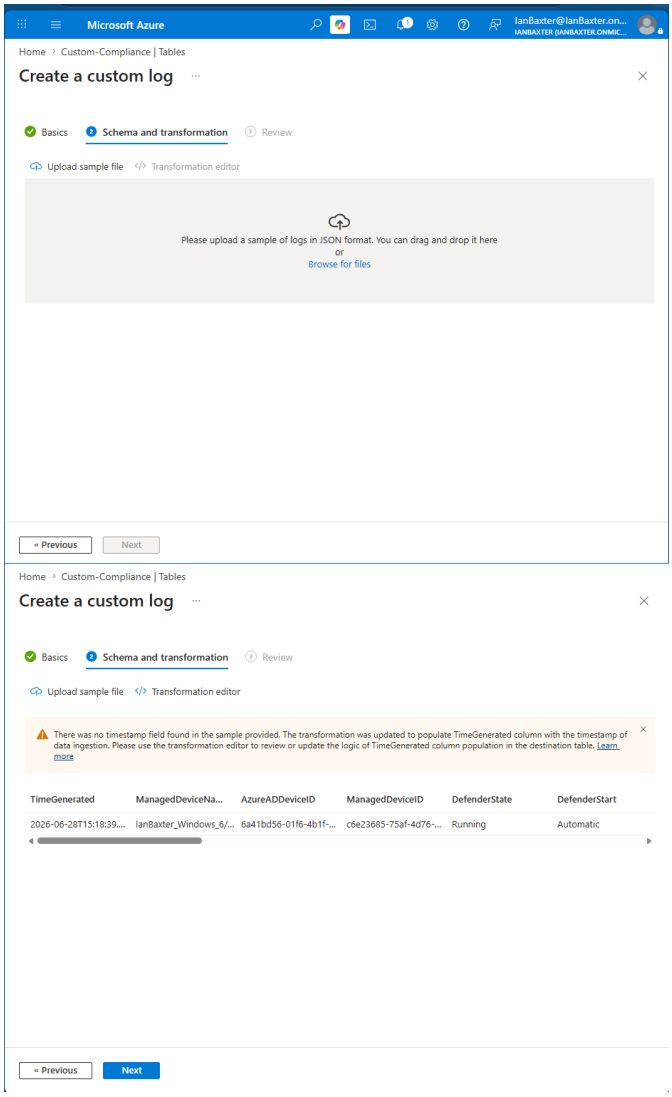


Upload the Json data you saved from the script test.

You should see the warning that a transform has been created for TimeGenerated. If your JSON doesn't contain the mandatory TimeGenerated data, a transform will be automatically created for you.

You will also see all the names of values in the JSON you uploaded.

Click "Next" and "Continue".

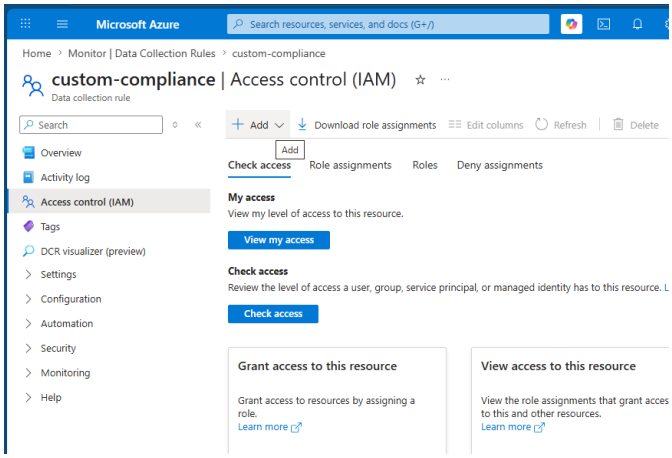


Permission the Service Principal

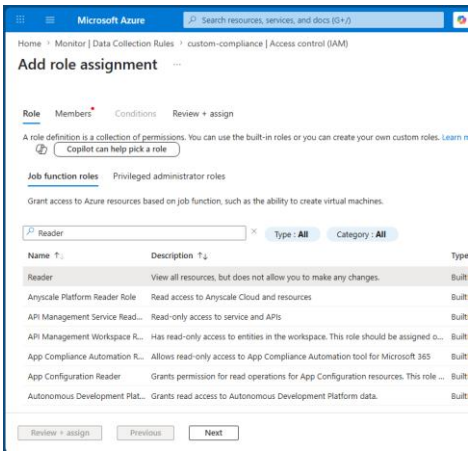
DCR

In the Azure Portal, search for “Data Collection Rule” and launch the applet.

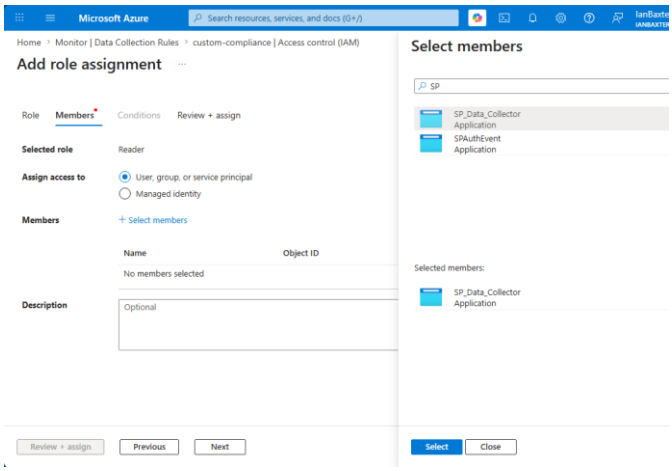
Select “Access Control” and click “Add”. “Add Role Assignment”.



Search for and select the “Reader” role. Click “Next”.

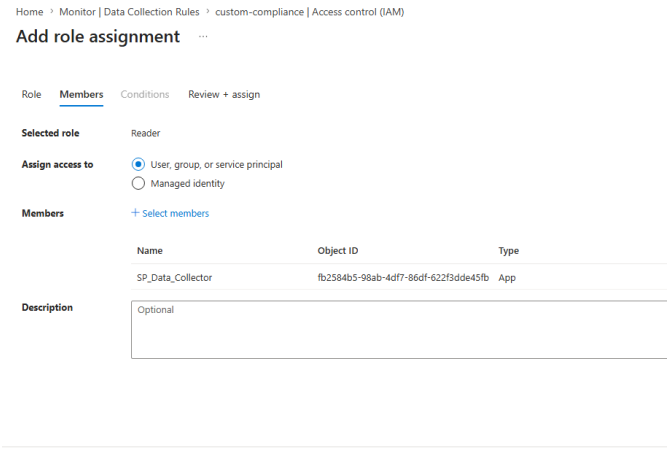


Click “Add Members”, search for and select the Service Principal account and click the “Select” button.

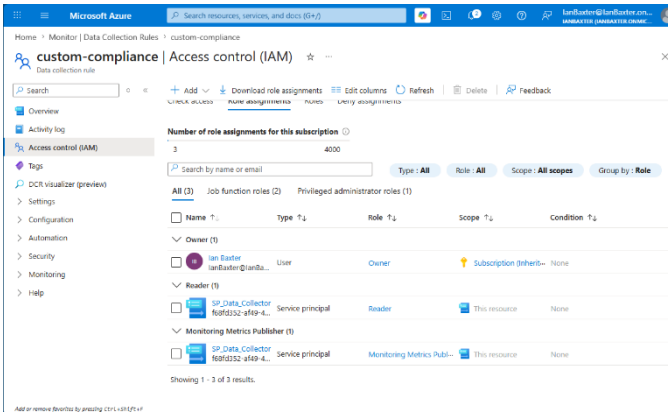


Click “Review and Assign”.

Repeat these steps to add the “Monitoring Metrics Publisher” role.



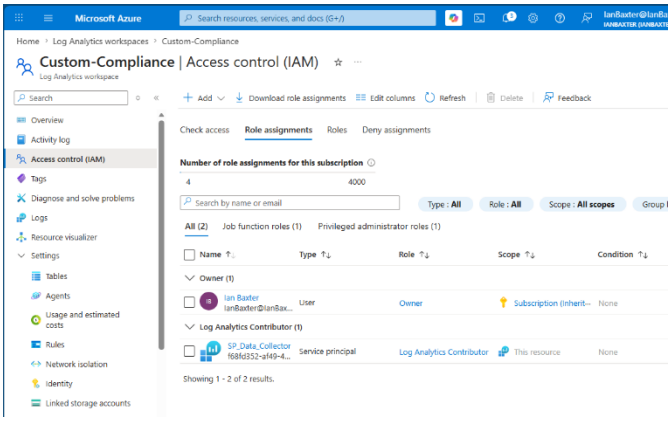
Verify the roles have been properly assigned.



Workspace

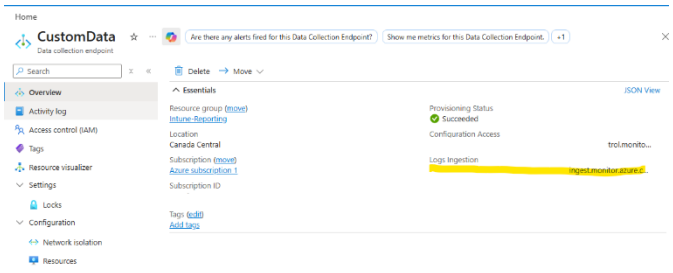
Search “Log Analytics Workspaces” in the Portal, load the workspace you created.

Add the “Log Analytics Contributor” role to the Service Principal.



Update the script for the DCE and DCR and test

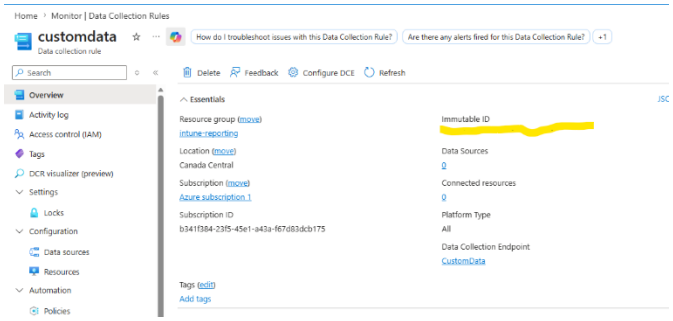
Copy the Logs Ingestion URL from the DCE Overview.



Paste the URL into the configuration block in the script.

```
#
# Log Analytics Data (for JSON submission)
$DcrImmutableId = "<DCR Immutable ID>"
$DceURI = "<DCE Data Ingestion URL>"
$streamName = "<Stream Name from DCR JSON>"
#
# Script logging
```

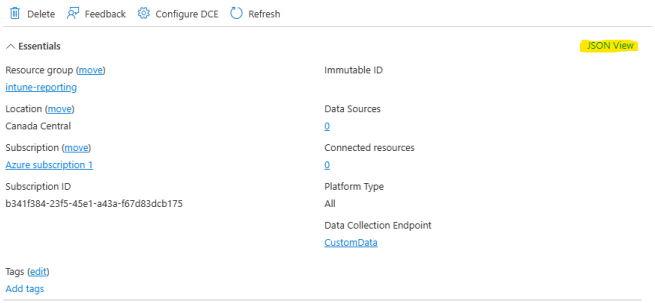
Copy the DCR Immutable ID from the DCR Overview.



Paste the ID into the configuration block of the script.

```
#
# Log Analytics Data (for JSON submission)
$DcrImmutableId = "<DCR Immutable ID>"
$DceURI = "<DCE Data Ingestion URL>"
$streamName = "<Stream Name from DCR JSON>"
#
```

In the DCR Overview, click the “JSON View” link.



Look for this section of the code and copy the stream name.

```
    "dataCollectionEndpointId": "/subscriptions/b341f384-23f5-45e1-a43a-f67d8
    "streamDeclarations": {
        "Custom-CustomData_CL": {
            "columns": [
                {
                    "name": "TimeGenerated",
                    "type": "datetime"
                },
```

Paste the stream name into the correct line of the script.

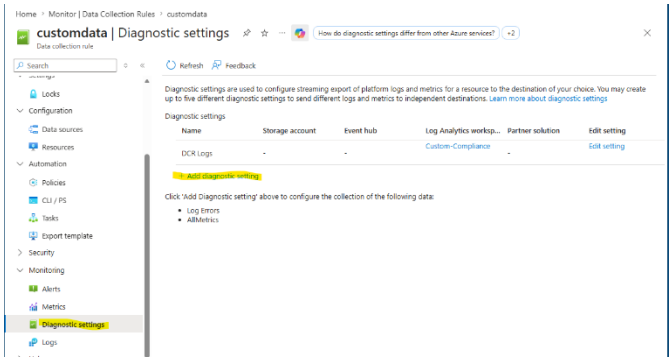
```
11  #
12  # Log Analytics Data (for JSON submission)
13  $DcrImmutableId = "<DCR Immutable ID>"
14  $DceURI = "<DCE Data Ingestion URL>"
15  $streamName = "<Stream Name from DCR JSON>"
16  #
17  # Script logging
```

Make sure you have a log file path specified. Given Intune's console ability to collect logs remotely, this can be very useful.

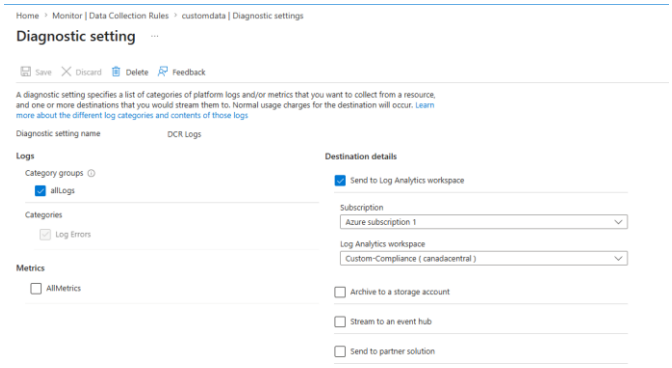
```
15  $streamName = "<Stream Name from DCR JSON>"
16  #
17  # Script logging
18  $LogFile = "<Log File Path>"
19  #
20  # If a log file has been specified and it already exists, d
21  if (-not [string]::IsNullOrWhiteSpace($LogFile) -and (Test-
```

Besides the log, Diagnostic Settings in a DCR can also help with debugging.

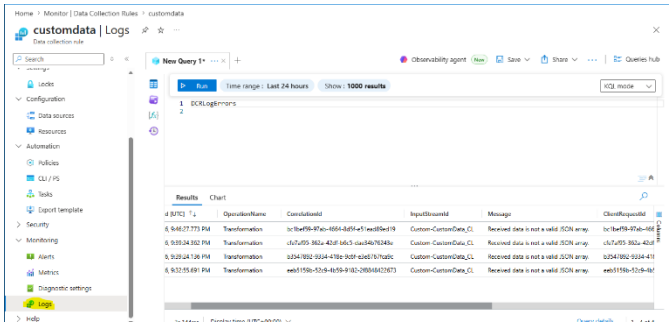
Load the DCR, select "Diagnostic Settings" in the left menu and click "Add Diagnostic Setting…"



Configuring these options will send all the DCR logs to the Log Analytics workspace.



Selecting Logs in the left pane and clicking the "Run" button will show you any found issues.

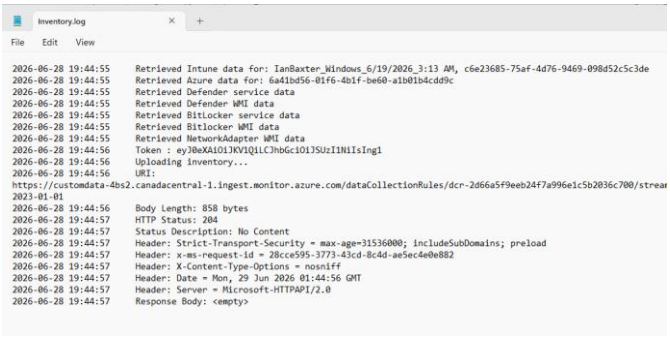


If the configuration is correct, you can run the script and get a success message.

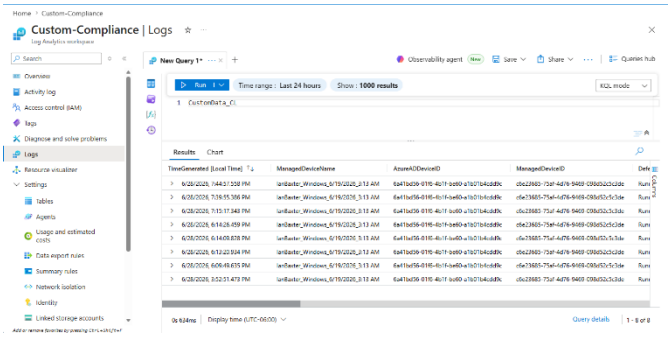
```
PS C:\WINDOWS\system32> E:\My Drive\Repo\Intune Lab\Projects\Custom Reporting SP\Custom_Inventory.ps1
InventoryDate:28-06 19:39 DeviceInventory:OK (204)

PS C:\WINDOWS\system32> |
```

If not, the logs should show you where you went wrong.



If it went right, the data can be viewed in the Log Analytics table.



Remove the manually deployed Certificate

Deploy the Private Key to all devices

Create the Custom Compliance Policy and Deploy

Verify Data is being collected